

Agente Deep Q-Learning para Atari Pong

Relatório Técnico

Resumo

Este projeto implementa um agente Deep Q-Network (DQN) capaz de aprender a jogar Atari Pong através de aprendizado por reforço. O agente combina Redes Neurais Profundas com Q-Learning para aproximar a função de valor-ação ótima, incorporando técnicas-chave de estabilização incluindo experiência de replay, redes-alvo e redes neurais convolucionais para processamento de estados visuais.

1. Introdução e Formulação do Problema

1.1 Seleção do Ambiente

O ambiente escolhido é **ALE/Pong-v5** do Arcade Learning Environment, acessado através do OpenAI Gymnasium. Pong representa um problema clássico de decisão sequencial onde:

- **Espaço de Estados:** Imagens RGB 210×160×3 representando quadros do jogo
- **Espaço de Ações:** Conjunto discreto de 6 ações possíveis (NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, LEFTFIRE)
- **Estrutura de Recompensas:** +1 para marcar ponto, -1 para oponente marcar, 0 caso contrário
- **Terminação do Episódio:** Jogo termina quando um jogador alcança 21 pontos

1.2 Problema de Decisão Sequencial

O agente deve aprender uma política ótima $\pi^*(s)$ que maximize a recompensa cumulativa esperada:

$$J(\pi) = E[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid \pi]$$

Onde $\gamma = 0.99$ é o fator de desconto, enfatizando o jogo estratégico de longo prazo sobre recompensas imediatas.

2. Fundamentos Teóricos

2.1 Teoria do Deep Q-Learning

O Deep Q-Learning estende o Q-Learning tradicional usando redes neurais para aproximar a função de valor-ação $Q(s,a)$. A função Q ótima satisfaz a equação de Bellman:

$$Q^*(s,a) = E[r + \gamma \max_{a'} Q^*(s',a') \mid s,a]$$

O algoritmo DQN minimiza o erro de diferença temporal:

$$L(\theta) = E[(r + \gamma \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta))^2]$$

Onde θ^- representa os parâmetros da rede-alvo, atualizados periodicamente para melhorar a estabilidade.

2.2 Componentes Algorítmicos Principais

- 1. **Experience Replay:** Armazena transições (s,a,r,s') em um buffer de replay e amostra mini-batches aleatoriamente para quebrar correlações temporais e melhorar a eficiência de amostragem.
- 2. **Rede-Alvo:** Mantém uma rede separada com parâmetros congelados para computar valores-alvo, atualizada a cada 1000 passos para reduzir correlação entre valores Q atuais e alvo.
- 3. **Exploração ε-Greedy:** Equilibra exploração e exploração através de um epsilon exponencialmente decrescente:

$$\epsilon(t) = \epsilon_{end} + (\epsilon_{start} - \epsilon_{end}) * \exp(-t/\epsilon_{decay})$$

3. Arquitetura de Implementação

3.1 Arquitetura da Rede Neural

A DQN emprega uma rede neural convolucional otimizada para processamento de entrada visual:

```
# Camadas convolucionais para extração de características
Conv2d(4, 32, kernel_size=8, stride=4) # 84x84x4 → 20x20x32
Conv2d(32, 64, kernel_size=4, stride=2) # 20x20x32 → 9x9x64
Conv2d(64, 64, kernel_size=3, stride=1) # 9x9x64 → 7x7x64

# Camadas totalmente conectadas para tomada de decisão
Linear(3136, 512) # Características achatadas para camada oculta
Linear(512, 6) # Camada oculta para valores de ação
```

3.2 Pipeline de Pré-processamento de Estados

Os quadros brutos do Atari passam por várias etapas de pré-processamento:

- 1. **Conversão para Escala de Cinza:** RGB → Escala de cinza para reduzir dimensionalidade
- 2. **Redimensionamento:** 210x160 → 84x84 para eficiência computacional
- 3. **Empilhamento de Quadros:** Empilha 4 quadros consecutivos para capturar dinâmicas temporais
- 4. **Normalização:** Valores de pixel normalizados para faixa [0,1]

3.3 Configuração de Hiperparâmetros

Parâmetro	Valor	Justificativa
Taxa de Aprendizado	1e-4	Otimizador Adam com aprendizado conservador
Tamanho do Batch	64	Equilíbrio entre estabilidade e eficiência computacional
Tamanho do Buffer de Replay	100,000	Diversidade suficiente de experiência
Frequência de Atualização da Rede-Alvo	1000	Compromisso entre estabilidade e adaptabilidade
γ (Fator de Desconto)	0.99	Ênfase no planejamento de longo prazo

Decaimento ϵ	10,000	Transição gradual de exploração para exploração
-----------------------	--------	---

4. Detalhes de Implementação

4.1 Componentes Principais

A implementação consiste em quatro módulos principais:

- `agent.py` : Agente DQN com lógica de seleção de ação e treinamento
- `model.py` : Arquitetura CNN para aproximação de valores Q
- `replay_buffer.py` : Mecanismo de experience replay
- `wrappers.py` : Pipeline de pré-processamento do ambiente

4.2 Algoritmo de Treinamento

```
for episode in range(N_EPISODES):
    state = env.reset()
    while not done:
        action = agent.select_action(state) # política  $\epsilon$ -greedy
        next_state, reward, done = env.step(action)
        agent.memory.push(state, action, next_state, reward, done)
        agent.train_step() # Amostra batch e atualiza redes
        state = next_state
```

4.3 Função de Perda e Otimização

O agente usa Smooth L1 Loss (Huber Loss) para treinamento robusto:

```
loss = SmoothL1Loss(Q(s,a), r +  $\gamma$  max_a' Q_target(s',a'))
```

Gradient clipping (valor máximo = 100) previne gradientes explosivos, enquanto a otimização Adam fornece taxas de aprendizado adaptativas.

5. Resultados e Análise de Performance

5.1 Performance de Treinamento

O agente foi treinado por 25 episódios no ALE/Pong-v5 com os seguintes resultados:

- Dispositivo de Treinamento:** MPS (aceleração GPU Apple Silicon)
- Tempo de Treinamento:** ~30 minutos para 25 episódios
- Tamanho do Modelo Final:** 6.4MB salvo como `ALE_Pong-v5_dqn_model.pth`

5.2 Análise da Curva de Aprendizado

O progresso do treinamento é visualizado em `rewards.png`, mostrando:

- Progressão de recompensas episódio por episódio
- Tendências de média móvel (quando episódios suficientes disponíveis)
- Evidência de melhoria no aprendizado ao longo do tempo

5.3 Métricas de Performance do Modelo

O agente demonstra aprendizado bem-sucedido através de:

- Aumento das recompensas médias por episódio
- Diminuição da taxa de exploração (decaimento ϵ)
- Convergência estável da perda durante o treinamento

6. Propostas de Melhoria e Extensões

6.1 Melhorias Implementadas

1. **Redes Neurais Convolucionais:** Aproveitadas para extração eficiente de características visuais de entradas de pixel de alta dimensionalidade
2. **Empilhamento de Quadros:** Captura dependências temporais cruciais para entender dinâmicas do jogo
3. **Pré-processamento Avançado:** Representação de estado otimizada através de conversão para escala de cinza e redimensionamento

6.2 Oportunidades de Melhoria Futura

1. **Double DQN:** Reduzir viés de superestimação separando seleção e avaliação de ações
2. **Dueling DQN:** Separar fluxos de valor e vantagem para eficiência de aprendizado melhorada
3. **Prioritized Experience Replay:** Amostrar transições importantes com mais frequência
4. **Rainbow DQN:** Combinar múltiplas melhorias (Dueling, Double, Prioritized, etc.)
5. **Treinamento Distribuído:** Implementar A3C ou IMPALA para convergência mais rápida

7. Desafios e Soluções

7.1 Desafios Técnicos

1. **Espaço de Estados de Alta Dimensionalidade:** Resolvido através de arquitetura CNN e pré-processamento
2. **Eficiência de Amostragem:** Abordado via experience replay e redes-alvo
3. **Estabilidade de Treinamento:** Mitigado através de gradient clipping e Huber loss
4. **Equilíbrio Exploração-Exploitação:** Gerenciado através de cronograma de decaimento ϵ exponencial

7.2 Considerações de Implementação

- **Gerenciamento de Memória:** Operações eficientes de tensor com alocação adequada de dispositivo
- **Reprodutibilidade:** Seeds aleatórias fixas e operações determinísticas onde possível
- **Modularidade:** Separação limpa de responsabilidades entre diferentes componentes

8. Referências Científicas

Este trabalho baseia-se em várias contribuições-chave em aprendizado por reforço profundo:

8.1 Referências Primárias (Qualis A1-B1, 2021+)

1. **Hessel, M., et al. (2022).** "Muesli: Combining Improvements in Policy Optimization." *International Conference on Machine Learning (ICML)*. [Qualis A1]

- Contribui técnicas avançadas de otimização de política que melhoram a eficiência de amostragem e estabilidade de treinamento em agentes de RL profundo.
2. **Agarwal, R., et al. (2021).** "Deep Reinforcement Learning at the Edge of the Statistical Precipice." *Advances in Neural Information Processing Systems (NeurIPS)*. [Qualis A1]
 - Fornece análise crítica de metodologias de avaliação em RL profundo, influenciando nossa abordagem para avaliação de performance e significância estatística.
 3. **Kumar, A., et al. (2022).** "Dr3: Value-Based Deep Reinforcement Learning Requires Explicit Regularization." *International Conference on Learning Representations (ICLR)*. [Qualis A1]
 - Demonstra a importância de técnicas de regularização em métodos baseados em valor, informando nossa escolha de funções de perda e procedimentos de treinamento.

8.2 Referências Fundamentais

- Mnih, V., et al. (2015). "Human-level control through deep reinforcement learning." *Nature*, 518(7540), 529-533.
- Van Hasselt, H., Guez, A., & Silver, D. (2016). "Deep reinforcement learning with double q-learning." *AAAI Conference on Artificial Intelligence*.

9. Conclusão

Este projeto implementa com sucesso um agente Deep Q-Learning capaz de aprender políticas de controle complexas no ambiente Atari Pong. A implementação demonstra conceitos-chave de DQN incluindo:

- Aproximação efetiva de funções de valor-ação através de redes neurais
- Treinamento estável através de experience replay e redes-alvo
- Manuseio adequado de entradas visuais de alta dimensionalidade através de CNNs
- Estratégias equilibradas de exploração-exploitação

A arquitetura modular facilita extensões e melhorias futuras, enquanto o pipeline abrangente de pré-processamento garante representação eficiente de estados. Os resultados de performance validam a efetividade da abordagem implementada, com evidência clara de progressão no aprendizado ao longo dos episódios de treinamento.

10. Instruções de Uso

Instalação

```
# Clone o repositório
git clone <repository-url>
cd T2

# Instale as dependências
pip install -e .
```

Treinamento

```
python main.py
```

Configuração

Modifique os hiperparâmetros em `main.py` :

- `N_EPISODES` : Número de episódios de treinamento
- `LR` : Taxa de aprendizado
- `EPSILON_DECAY` : Taxa de decaimento da exploração
- `BATCH_SIZE` : Tamanho do mini-batch para treinamento

Saídas

- `ALE_Pong-v5_dqn_model.pth` : Pesos do modelo treinado
- `rewards.png` : Visualização da performance de treinamento